

Streaming i DLT

Porównanie

Spark Structured Streaming

Zbudowany na silniku Spark SQL

Oparty na **Tabeli**

Dataset/DataFrame API

<https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>

[Structured Streaming - Azure Databricks | Microsoft Docs](#)

Spark Streaming

Rozszerzenie do Spark API

Oparty na **Micro-batch**

DStream (stream bloków RDD)

<https://spark.apache.org/docs/latest/streaming-programming-guide.html>

Note

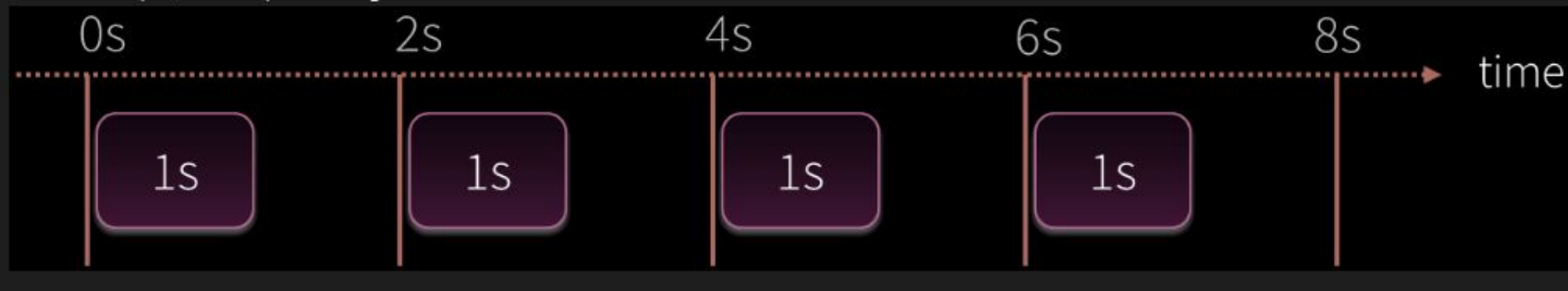
Spark Streaming is the previous generation of Spark's streaming engine. There are no longer updates to Spark Streaming and it's a legacy project. There is a newer and easier to use streaming engine in Spark called Structured Streaming. You should use Spark Structured Streaming for your streaming applications and pipelines. See [Structured Streaming Programming Guide](#).

Przetwarzanie micro-batcha

Dla każdego przedziału czasowego przetwarzamy dane z poprzedniego przedziału czasowego (np ostatnie dwie sekundy)

W tym przykładzie przetwarzamy dane z dwóch sekund w jedna sekunde

In our example, we are processing two seconds worth of data in about one second.



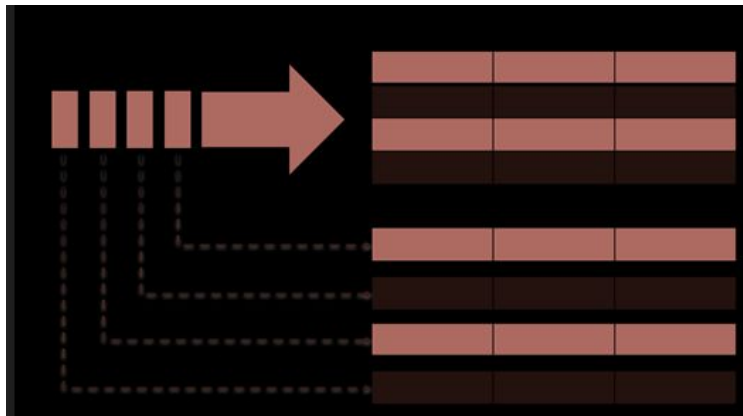
Mikro-batch alternatywa

Co się stanie jeśli nie przetwarzamy danych wystarczająco szybko ?

CEL: przetworzenie danych z poprzedniego przedziału zanim wpłyną kolejne dane

Rozwiązanie: Spark Structure Streaming traktuje stream (micro batch) jako **tabele** i ciągle dodaje (**append**)

Trigger interval = append (new row)



Streaming DLT

- **Streaming jest 'stateful'**

- Zapewnia **jednorazowe przetwarzanie wierszy danych**
- Dane wejściowe są wczytywane tylko raz - oszczędność czasu

```
CREATE STREAMING LIVE TABLE tbl AS SELECT sum(column) FROM  
prod.table
```

- Wyliczają wyniki wykonując operacje append (dodają) stream from Kafka lub kinesis lub [Autoloadera](#)
- Redukują koszty i latencje - przetwarza tylko nowe dane

DLT

- Jest to zarządzana usługa do ETL który używa **prostego stylu deklaratywnego** do budowanie pipeline'ów.
- **Automatyczne wykrywanie zależności** między tabelami / widokami i uruchamianie wykonanie zadań w odpowiedniej kolejności.
- Język deklaratywny -> **definiujesz co ma być zrobione** a nie jak
- Automatycznie wygeneruje **lineage** (historię przetwarzania)
- Są zintegrowane z **lakehousem**, repozytorium **unity catalog** i **workflow**.
- Wbudowana kontrola jakości

Delta Live Tables

1. Definiujesz w notatniku

- kod trzymasz w repo

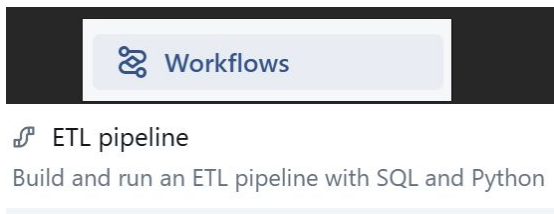
2. Tworzysz pipeline (workflow)

- Może on składać się z wielu notatników, konfiguracji

3. Job Start

- DLT stworzy bądź zaktualizuje table

```
1 CREATE LIVE TABLE daily_stats
2 AS SELECT sum(rev) - sum(costs) AS profits
3 FROM prod_data.transactions
4 GROUP BY day
```



Delta Live Tables

- Zdefiniowana przez kwerendę SQL
- DLT są zmaterializowanymi widokami dla Lakehouse
- Stworzona przez pipeline

```
CREATE LIVE TABLE table_name
AS
SELECT kolumna, kolumna2
FROM prod.table_name
```

Live tables provides tools to:

- Manage dependencies
- Control quality
- Automate operations
- Simplify collaboration
- Save costs
- Reduce latency

DLT Testowanie - Expectations

Expectations to opcjonalne klauzule dodawane do deklaracji DLT, które stosują kontrolę jakości danych **dla każdego rekordu** przechodzącego przez zapytanie. składa się z trzech rzeczy:

1. **Opis**, który działa jako unikalny identyfikator i umożliwia śledzenie metryk
2. **Instrukcja** logiczna, która zawsze zwraca TRUE lub FALSE
3. **Akcja**, którą należy wykonać, gdy rekord nie spełnia oczekiwań

Działanie	Wynik
Warn (domyślne)	Nieprawidłowe rekordy są zapisywane w miejscu docelowym ; niepowodzenie jest zgłaszane jako metryka dla zestawu danych
Drop	Nieprawidłowe rekordy są usuwane przed zapisaniem danych w miejscu docelowym; awaria jest zgłaszana jako metryka dla zestawu danych.
Fail	Nieprawidłowe rekordy uniemożliwiają pomyślną aktualizację. Przed ponownym przetworzeniem wymagana jest ręczna interwencja .